

CS & IT ENGINEERING



Data Structure & Programming

Linked Lists

Lecture No.- 03



By- Satya sir

Recap of Previous Lecture



SLH



- Creation
- Insertion
 - At Beginning : $O(1)$ Constant
 - At End : $O(n)$
 - At Given Position : $O(n)$ } Linear
- Deletion
 - First Node ✓ : $O(1)$
 - Last Node
 - middle Node } H/w

Topics to be Covered



- Deletion from SLL
 - last Node
 - middle Node
- SLL - Search, Access
- Doubly Linked List
 - Insertion



Topic : SLL – Deletion, Search



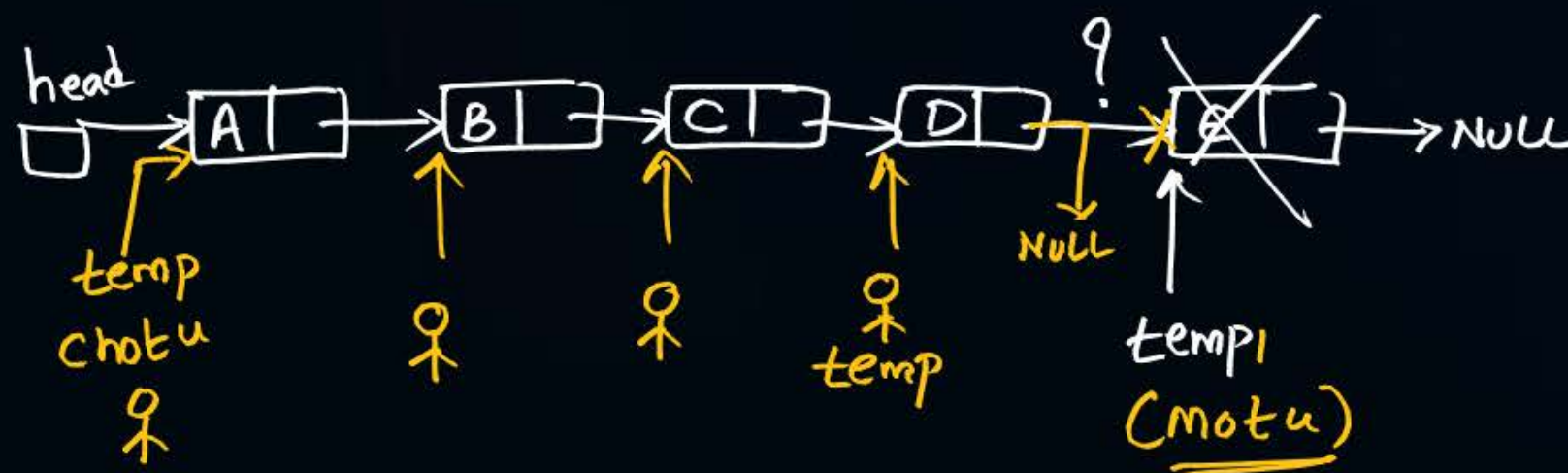
Deletion of Last Node

```
void Delete_last_SLL ( )
```

```
{  
    if (head == NULL)  
    {  
        printf("Empty list");  
        return;  
    }
```

1 Only one Node

```
if (head->next == NULL) {  
    struct SLL *temp = head, *temp1;  
    while (temp->next->next != NULL)  
    {  
        temp = temp->next;  
    }  
    temp1 = temp->next;  
    struct SLL *temp;  
    temp = head;  
    head = NULL;  
    free(temp);  
    return;  
}
```



temp->next = NULL

free(temp1);

}

Time Complexity : $O(n)$



Topic : SLL – Deletion, Search



Deletion of middle Node (At given Position)

```
Void delete_middle_SLL (int Position)
{
```

```
    if (head == NULL) // Empty list
        Print ("Empty list");
```

```
    else if (head->next == NULL) // Only one Element
```

```
    {
        Struct SLL *temp = head;
        head = NULL;
        free(temp);
    }
```

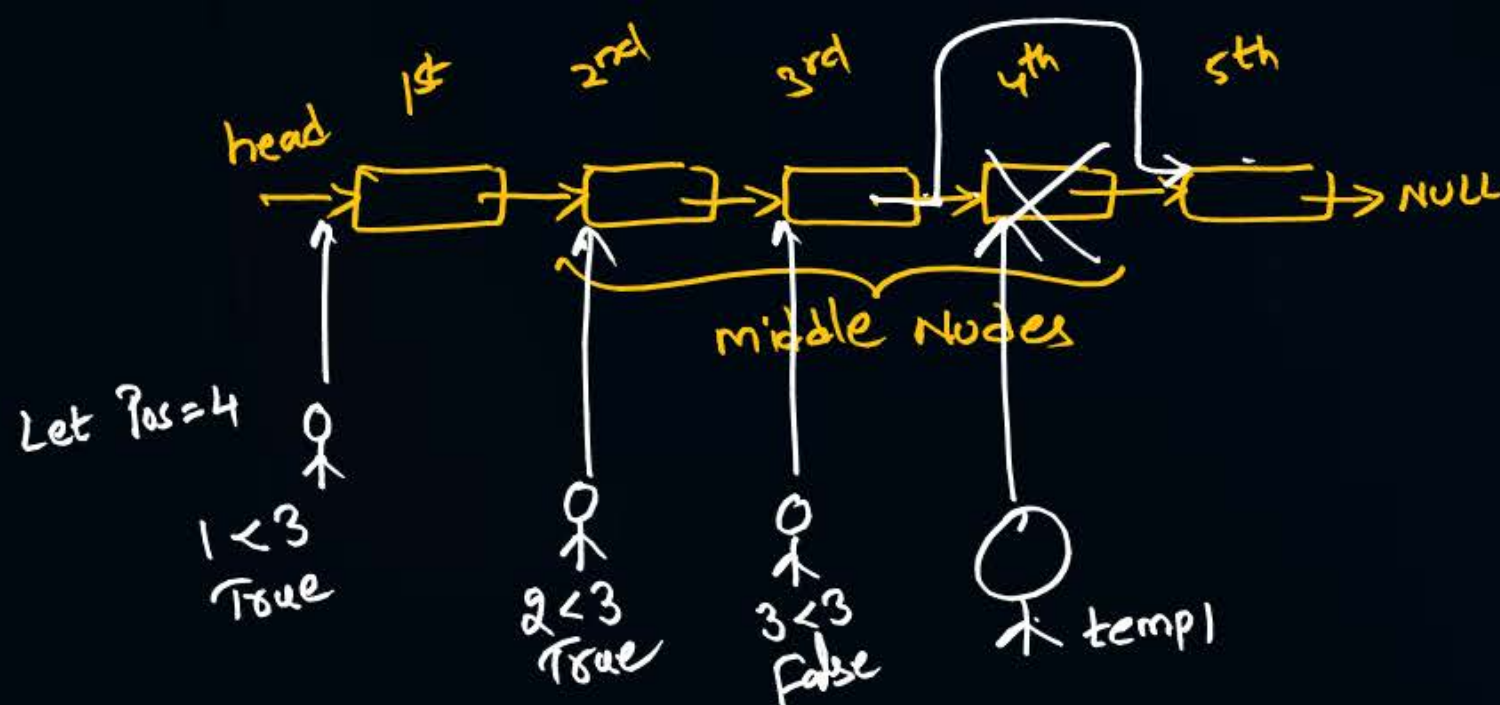
```
    else
```

```
    {
        Struct SLL *temp = head; *temp1;
        int Count = 1;
```

```
        while (Count < Position - 1)
        {
            temp = temp->next;
            Count++;
        }
```

```
        temp1 = temp->next;
```

```
        temp->next = temp->next->next; (or) temp->next = temp1->next;
        free(temp1);
```



Time Complexity = $O(n)$



Topic : SLL – Deletion, Search



Search an Element in SLL
(Key)

Best case : $O(1)$ first element is the key

Average case : $O(n)$ middle element is the key

Worst case : $O(n)$ Last element is the key (or)
key is not found.

Access/Traverse SLL

- Linked List support sequential access only.

- Hence, ^{Worst-case} Time complexity : $O(n)$

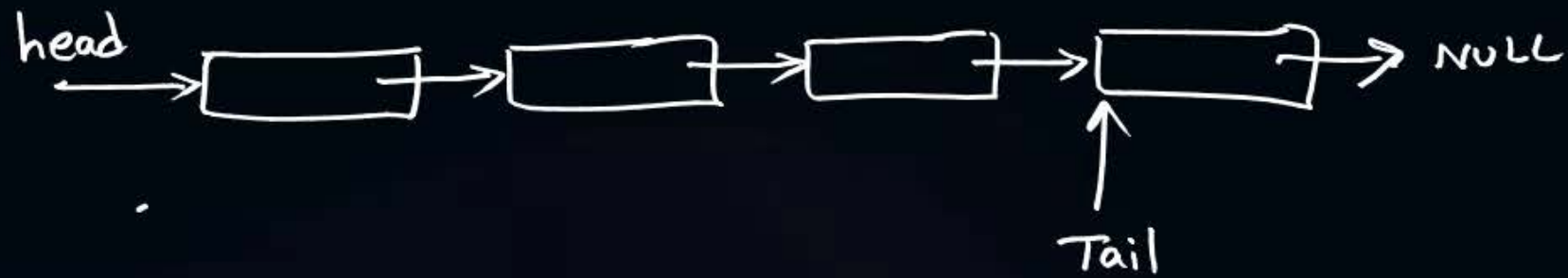


Topic : Doubly Linked List-1



SLL with both head and Tail Pointers

At first Node At last Node



Insertion:

- At Beginning : $O(1)$ $new \rightarrow next = head; head = new;$
- At middle : $O(n)$
(Second last Pos)
- At last : $O(1)$ $Tail \rightarrow next = new; new \rightarrow next = NULL;$
 $Tail = new;$

Search operation: Best case: Key = head element, Key = Tail element $O(1)$
 Avg case : key is in middle $\Rightarrow O(n)$
 & worst case or not found

Deletion

First Node : $O(1)$

middle Node : $O(n)$

Last Node : $O(n)$

Traverse/Access : $O(n)$



Topic : Doubly Linked List-1



Doubly Linked List

- Each Node Contains 2 links (Prev, next), Pointing to Previous and next nodes, In addition to data.



Create DLL Node :

```
struct DLL
{
    struct DLL *Prev;
    data declaration
    struct DLL *next;
} *head = NULL
```

Insertion / Deletion into / from DLL

- First Node
- middle Node
- Last Node



Topic : Doubly Linked List-1



Insert At Beginning into DLL

```
Void insert_Begin_DLL (int Value)
{
```

```
    struct DLL *new;
```

```
    new = (struct *) malloc (sizeof (struct DLL *));
```

```
    new->data = Value;
```

```
    if (head == NULL)
```

```
    {
        head = new;
        new->prev = NULL;
        new->next = NULL;
    }
```

```
    else
```

```
    {
```

```
        new->next = head; ✓
```

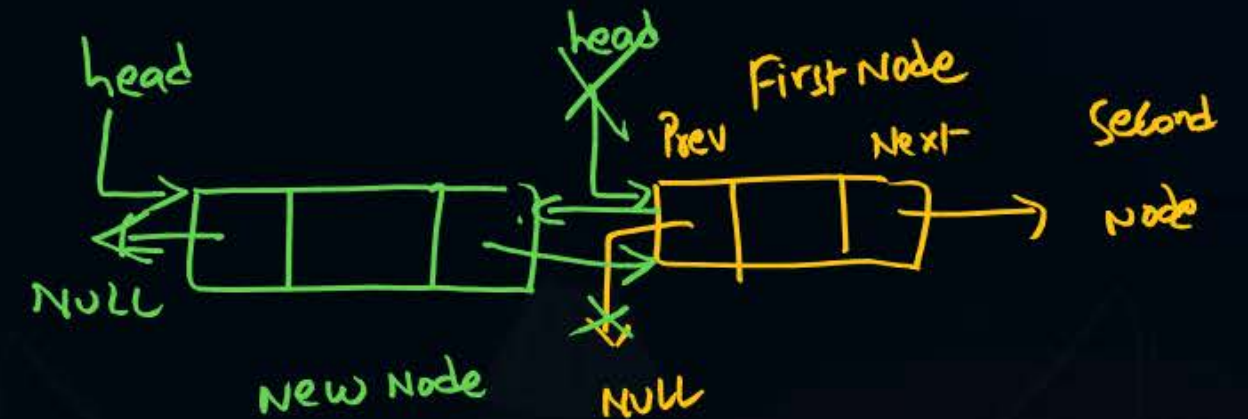
```
        new->next->prev = new; ✓
```

```
        new->prev = NULL; ✓
```

```
        head = new; ✓
```

```
    }
```

```
}
```



Time Complexity :

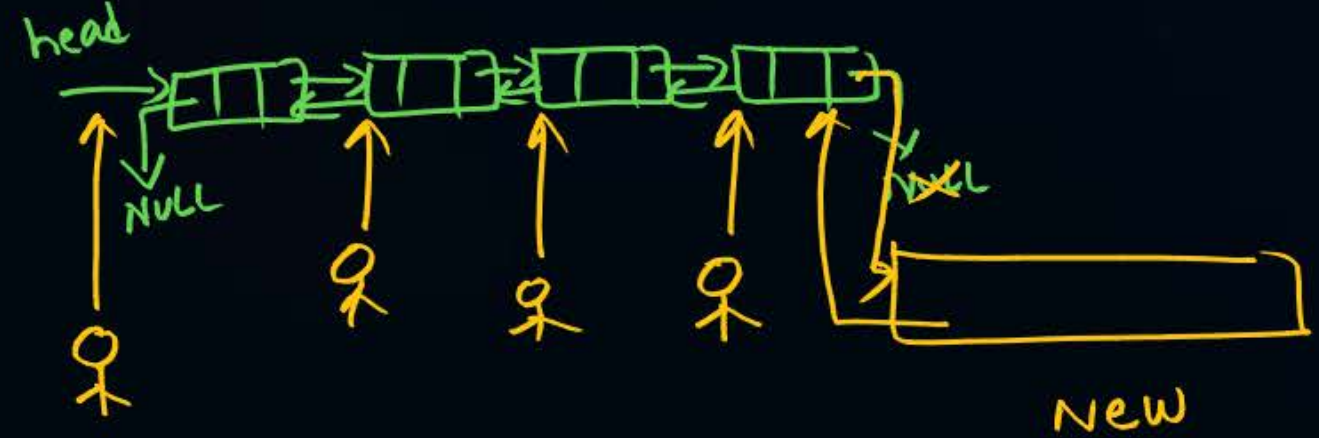
with only head } $O(1)$
with head & Tail }



Topic : Doubly Linked List-1



Insert At End into DLL $\begin{cases} \text{T.C} \Rightarrow O(n) \text{ without Tail} \\ O(1) \text{ with Tail} \end{cases}$



Void insert_End_DLL (int Value)

{

Struct DLL *new;

new = (Struct *) malloc (Size of (Struct DLL *));

new->data = Value;

if (head == NULL)

{

head = new;

new->prev = NULL;

new->next = NULL;

} // Tail = new;

else

{ struct DLL *temp = head;

new->next = NULL;

while (temp->next != NULL)

temp = temp->next;

temp->next = new;

new->prev = temp;

}

// If Tail Pointer is there

else

{

Tail->next = new;

new->prev = Tail;

new->next = NULL;

Tail = new;

}



Topic : Doubly Linked List-1



Insertion at middle of DLL

H/w

Deletion operation in DLL H/w



2 mins Summary



— SLL — Deletion

— DLL — Insertion

To be contd . . .



Next class on 25/Mar/2025 - Tuesday 7AM.



THANK - YOU